Labelling guide: who is commanding the dog?

Why we label

The robot decides by itself whether a gesture is meant for it, using fuzzy logic (not a trained network). To prove it works, we need **human answers to compare against**: for every gesture, was it meant for the dog or not? The paper then reports how often the robot obeyed a bystander by mistake and how often it obeyed the real commander.

So these labels are the **answer key** for testing. They don't train the robot, and the robot never needs them when it's working.

The one rule

Look at each moment and say WHO is gesturing to the dog. Being the most visible or the closest person doesn't count. A person half-hidden at the back who gestures to the dog is the commander. A big, clear gesture from the front person aimed at a friend is **not** for the dog.

The tool (use this one)

```
python label_moments.py takes/D1_S2_r1                 # annotator A (Mihara)
python label_moments.py takes/D1_S2_r1 --annotator B   # annotator B (second person)
```

Gestures happening at the same time are grouped into one **moment**. The clip plays once, then freezes:

- **Orange box with a big #ID** = this person is making a command gesture (fist, palm…).
- **Thin blue box** = other people (not gesturing).

Type the ID(s) of whoever is gesturing TO THE DOG, then press Enter:

You type	Meaning
`3`	#3 is commanding the dog; every other orange person in this moment is not
`3 5`	#3 and #5 both command the dog (scenario S5)
`0`	Nobody here is gesturing to the dog (all orange people are gesturing to each other, or by accident)
`3 x5`	#3 commands the dog; #5's orange box is a **bad detection** (no real hand, wrong person, box on two people)

Other keys: `p` = play the clip again, `Backspace` = fix typing, `b` = back one moment, `q` = save and quit (it remembers where you were).

The tool only accepts IDs that have an orange box. If you mistype, it tells you which IDs are allowed.

How to decide

1. During the gesture, is that person **facing and looking at the dog**? Yes → type their ID.
2. Check the **take log**: who was the commander, and which scenario was it?
3. In S4, the same person gestures to a friend and then to the dog. Those are different moments; judge each one by where they're facing at that moment.
4. Not sure even after `p`: don't type that ID (leave them as "not for the dog"), or mark the box as bad with `x` if the detection itself is wrong.

Scenario	Typical answer
S1: commander alone	commander’s ID
S2: bystanders gesture to each other	commander’s ID only (or `0` if only bystanders are gesturing in that moment)
S3: simultaneous gestures	commander’s ID; the bystander is automatically “not for the dog”
S4: commander turns to a friend	`0` while gesturing to the friend; commander’s ID when gesturing to the dog
S5: two people command the dog	both IDs, e.g. `2 4`
S6: bystanders walk past waving	commander’s ID if they gestured, else `0`
W1–W3: dog walking	same rules

Last question for each take

After the last moment, the terminal asks how many times someone **clearly gestured to the dog but never got an orange box** (for example, a hidden hand). Type the number (Enter = 0, `?` if unsure). A missed command still counts as a miss, so this keeps results honest.

Two labellers, separately

A and B label **every take without talking about it** and without opening each other’s files. We measure how much they agree.

All of one day:

```
for t in takes/D1_[SW]*; do python label_moments.py "$t"; done                  # A
for t in takes/D1_[SW]*; do python label_moments.py "$t" --annotator B; done    # B
```

Windows PowerShell:

```
Get-ChildItem takes -Directory -Filter "D1_[SW]*" | % { python label_moments.py $_.FullName
}
```

(add `--annotator B` for B).

Before and after

- **Before:** detection fix v3 applied (people every frame, hands tracked), `count_people.py` checked, and `python reprocess.py` run on all takes. Don’t reuse labels from `labels_old/`.
- **After:** `python check_dataset.py` until no problems. Then hand over the full data on the SSD and the small zip:

  ```
  python -c "import shutil,glob,os; [shutil.copytree(t, os.path.join('small',t),
  ignore=shutil.ignore_patterns('*.mp4','depth')) for t in glob.glob('takes/*')];
  shutil.make_archive('takes_small','zip','small')"
  ```

(The older one-clip-per-person tool, `label_episodes.py`, still works and saves the same files. Use one tool per take.)